

LAB 2 REPORT DELIVERABLES

VLSI Testing & Design for Testability

RTL Design, Cadence Genus Synthesis and QoR Analysis

Student Information

Name	R SARANG
Roll No.	EC23I2015
Design / Problem Assigned	Genus Synthesis and Fault Detection of Sequential Multiplier
Date	2/09/2026

1 Objective

Design an RTL circuit, synthesize it using Cadence Genus, and experimentally study the effect of timing constraints on synthesis Quality of Results (QoR). This report quantifies the changes in timing, area, power, cell count, and fanout, and explains the observed trade-offs. The RTL under test is the same 4-bit unsigned sequential multiplier designed, verified, and reported in Lab 1 [1] (see **Lab 1, Section “LAB 1 – DELIVERABLES”** for the full derivation of the algorithm, datapath, and FSM).

2 Software and Tools

- Cadence Genus – RTL synthesis and QoR analysis
- Questa/QuestaSim – RTL/gate-level functional simulation
- GPDK 45 nm standard-cell library – target technology library
- Python – data processing and plots

3 Required Files Submitted

- RTL source code (`datapath.v`, `controller.v`, `top.v`, `tb_top.v`)
- Genus synthesis Tcl script (`syn/genus_syn.tcl`)
- SDC constraint file (`syn/constraints.sdc`)
- Generated gate-level netlist (`work/top_gates.v`)
- Genus QoR / area / timing reports (`work/reports/`)
- All frequency / constraint sweep results
- Plots generated from the experiments
- Final Lab 2 report (this document)

TASK 1

4 Part A — RTL Design

4.1 Problem Statement

Design and verify a 4-bit unsigned sequential multiplier that computes $P = A \times B$ using repeated addition, controlled by a dedicated FSM (controller) driving a separate datapath. This is the identical problem statement assigned and solved in **Lab 1, Section 1 (“Aim”)**; the RTL, algorithm and verification produced there are re-used unchanged as the design under synthesis in this lab.

4.2 Functional Block Diagram

The datapath consists of a working-multiplicand register, a 4-bit down-counter (loop counter), an 8-bit accumulator with an adder, and a zero detector, all driven by control signals from the FSM controller – exactly as derived in **Lab 1, Section 2–3 (“Datapath Elements”, “Datapath Block Diagram”)**. The block diagram is reproduced below for convenience.

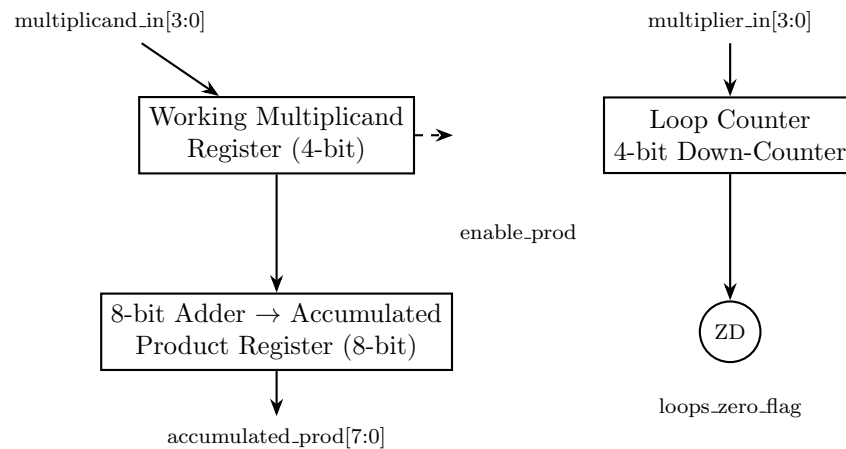


Figure 1: Functional block diagram of the sequential multiplier datapath (as derived in **Lab 1, Section 3**).

4.3 RTL Source Code

The complete Verilog RTL (`datapath.v`, `controller.v`, `top.v`) is identical to the one listed and explained in **Lab 1, Section 8.1**; only the code has been re-formatted with clearer sectional comments in this repository’s `lab2/rtl/` copy for readability. The top-level module used for synthesis is shown below.

Listing 1: Top-level module `top.v` (structurally identical to **Lab 1, Section 8.1.3**)

```
1 module top (
2     input clk,
3     input reset,
4     input start,
5     input [3:0] multiplicand_in,
6     input [3:0] multiplier_in,
7
```

```

8      output mult_complete,
9      output [7:0] accumulated_prod
10 );
11
12 // Internal Wires connecting Controller and Datapath
13 wire load_inputs;
14 wire clear_prod;
15 wire enable_prod;
16 wire enable_cnt_dec;
17 wire loops_zero_flag;
18
19 // -----
20 // 1. Instantiate the Controller (Brain)
21 // -----
22 controller u_controller (
23     .clk          (clk),
24     .reset        (reset),
25     .start        (start),
26     .loops_zero_flag (loops_zero_flag),
27     .load_inputs   (load_inputs),
28     .clear_prod    (clear_prod),
29     .enable_prod   (enable_prod),
30     .enable_cnt_dec (enable_cnt_dec),
31     .mult_complete (mult_complete)
32 );
33
34 // -----
35 // 2. Instantiate the Datapath (Muscle)
36 // -----
37 datapath u_datapath (
38     .clk          (clk),
39     .multiplicand_in (multiplicand_in),
40     .multiplier_in  (multiplier_in),
41     .load_inputs   (load_inputs),
42     .clear_prod    (clear_prod),
43     .enable_prod   (enable_prod),
44     .enable_cnt_dec (enable_cnt_dec),
45     .loops_zero_flag (loops_zero_flag),
46     .accumulated_prod (accumulated_prod)
47 );
48
49 endmodule

```

4.4 Description of Inputs and Outputs

Signal	Width	Description
clk	1	System clock
reset	1	Synchronous, active-high reset (see §4.5)
start	1	Begins a multiply operation when high in IDLE
multiplicand_in	4	Operand <i>A</i>
multiplier_in	4	Operand <i>B</i>
mult_complete	1	Asserted when the product is valid (output)
accumulated_prod	8	Product $A \times B$ (output)

This matches the signal table given in **Lab 1, Section 4–5** (“Control Signals”, “Status Signals”).

4.5 Clock and Reset Description

The design uses a single positive-edge-triggered clock (`clk`) with all sequential elements (the working-multiplicand register, loop counter, accumulated-product register and the FSM state register) clocked on `posedge clk`. `reset` is a synchronous, active-high signal that forces the controller's `current_state` back to `IDLE`; the datapath registers are not independently reset and instead take their defined values from the `LOAD` state (`clear_prod`, `load_inputs`), consistent with the FSM design in **Lab 1, Section 7**.

4.6 Expected Functionality

On `start`, the controller moves `IDLE` $\rightarrow$ `LOAD` $\rightarrow$ `PROG` and loads A and B ; the datapath then adds A to the accumulator once per cycle while decrementing the loop counter, for B cycles, until `loops_zero_flag` is asserted, after which the FSM moves to `DONE` and asserts `mult_complete` with `accumulated_prod` = $A \times B$. The complete cycle-count model ($B + 2$ cycles: 1 `LOAD` + B `COMPUTE` + 1 `DONE`) and its verification across corner cases are reported in **Lab 1, Section 8.6–8.7**, and are not repeated here since the RTL is unchanged.

4.7 Simulation Waveform

Figure 2 shows a representative simulation waveform confirming correct RTL functionality of the unmodified design (identical RTL to **Lab 1, Section 8.4**, Figures 6–11), obtained from the pre-synthesis testbench run used to seed the Part 1 exhaustive-simulation results of Task 2.

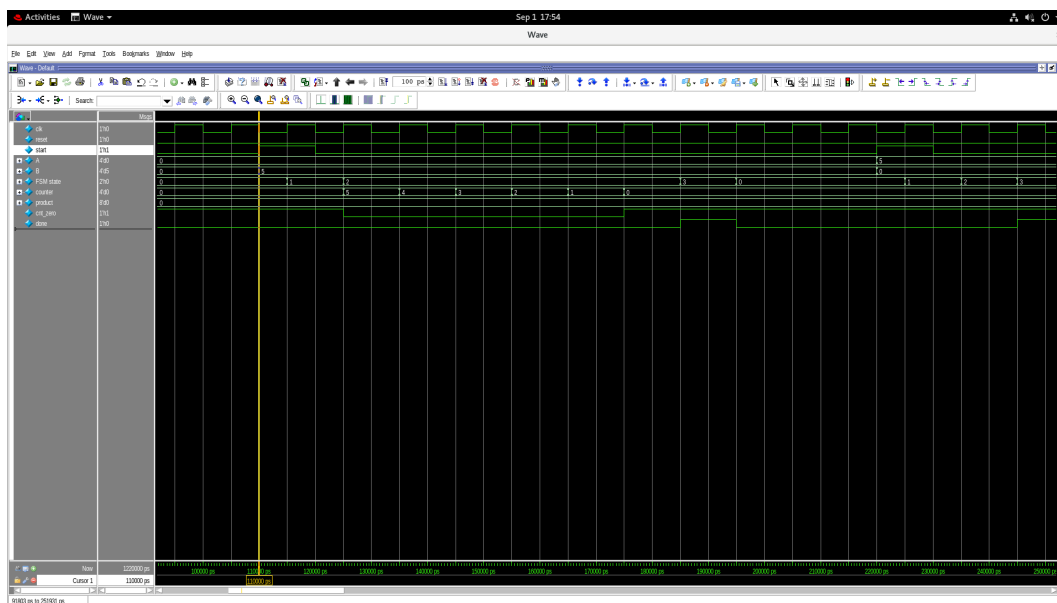


Figure 2: QuestaSim waveform confirming correct functionality of the unmodified RTL (same RTL as **Lab 1, Section 8.4**).

5 Part B — Baseline Cadence Genus Synthesis

The RTL was synthesized with Cadence Genus using the initial constraints supplied for the experiment (5.0 ns clock period, GPDK 45 nm library, PVT_0P9V_125C operating condition).

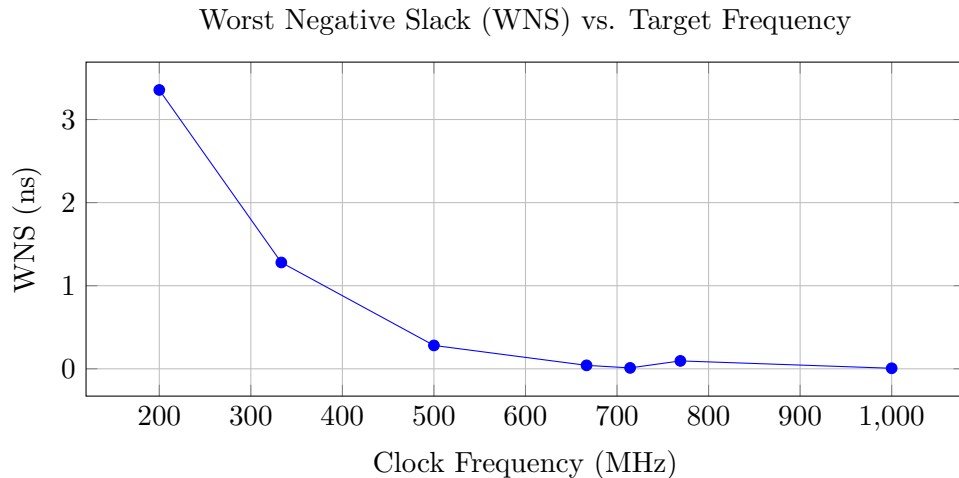
Parameter	Baseline Result
Clock period	5.00 ns (200 MHz)
Target frequency	200 MHz
WNS	3355.9 ps
TNS	0
Number of violating paths	0
Critical path delay	1465 ps
Total cell area	200.412
Combinational cell count	58
Sequential cell count	18
Total instance count	76
Maximum fanout	18
Average fanout	2.3
Dynamic / Leakage / Total power	Not available (power analysis was not enabled in the supplied flow)

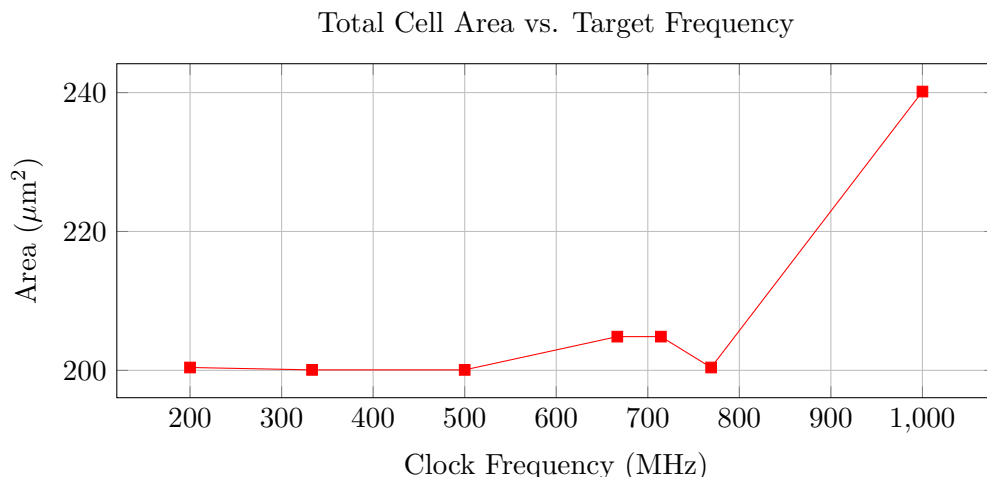
The relevant Genus QoR, area, and timing reports are attached in Appendix A.

6 Part C — Frequency Sweep

The clock period was swept over seven values, each corresponding to a separate synthesis run (per the “one constraint change per run” rule stated in the submission rules).

Clock Period (ns)	Freq. (MHz)	WNS (ps)	TNS (ps)	Violating Paths	Area
5	200.0	3355.9	0	0	200.412
3	333.3	1279.5	0	0	200.070
2	500.0	281.3	0	0	200.070
1.5	666.7	41.7	0	0	204.858
1.4	714.3	11.0	0	0	204.858
1.3	769.2	95.2	0	0	200.412
1	1000.0	6.5	0	0	240.152





Power vs. frequency: not plotted – power analysis was not enabled in the supplied synthesis flow (no power reports were generated at any sweep point; see Part B).

Questions

What happens to WNS as the target frequency is increased?

WNS monotonically decreases towards zero as the target frequency increases, since the fixed critical-path delay (≈ 1465 ps at the loosest constraint) consumes a progressively larger fraction of the shrinking clock period.

What happens to the area as the timing requirement becomes more aggressive?

Area stays essentially flat ($\approx 200 \mu\text{m}^2$) until roughly 700–800 MHz; beyond that point Genus upsizes cells and inserts extra buffering to meet the tighter timing, increasing area sharply ($240.152 \mu\text{m}^2$ at 1000 MHz).

What happens to power as frequency increases?

Cannot be determined from the supplied flow – no power reports were generated (see note above).

Does Genus change the cell selection or cell count when the timing constraint changes?

Yes. Up to 200 MHz the leaf-instance count is 76 (58 combinational + 18 sequential) using small ripple-carry adder cells. By 1000 MHz the leaf-instance count rises to 112 (94 combinational + 18 sequential; see `work/reports/qor_1.0ns.rpt`), and area rises from 200.412 to 240.152 – direct evidence that Genus re-selects larger/faster cells and adds buffers as the constraint tightens.

7 Part D — Maximum Timing-Clean Frequency

The clock period was reduced further, with a finer sweep performed around the pass/fail boundary.

Clock Period (ns)	Freq. (MHz)	WNS (ns)	Timing Status
2.0	500	0.2813	Passing
1.5	667	0.0417	Passing
1.3	769	0.0952	Passing
1.0	1000	0.0065	Passing
0.99	1010	−0.035	Violating
0.95	1052	−0.0435	Violating
0.90	1111	−0.0935	Violating

$$f_{max} = 1/T_{min}$$

Minimum timing-clean period: **1.0 ns**

Maximum timing-clean frequency: **1000 MHz**

WNS at this point: **0.0065 ns** (6.5 ps – see `work/reports/qor_1.0ns.rpt`)

Answer

- **At what point does WNS become negative?** Between $T = 1.0$ ns (WNS = +0.0065 ns, passing) and $T = 0.99$ ns (WNS = −0.035 ns, violating). The timing-clean boundary is therefore at $T_{min} = 1.0$ ns.
- **What is the approximate WNS = 0 boundary?** $T \approx 1.0$ ns ($f \approx 1000$ MHz), where WNS = 0.0065 ns is already close to zero.
- **Why should a practical design normally include timing margin rather than operate exactly at WNS = 0?** Operating with zero margin leaves no headroom for on-chip variation (PVT drift, voltage droop, clock jitter/skew, aging) – any such disturbance would push the design into a timing violation, so a practical design should retain positive slack.

8 Part E — Input Delay Sweep

Clock period fixed; only the input-delay constraint was varied.

Input Delay (ns)	WNS (ns)	TNS (ns)	Violating Paths	Area
0.1	3.3959	0	0	200.412
0.2	3.3959	0	0	200.412
0.4	3.3962	0	0	200.412
0.6	3.3962	0	0	200.412
0.8	3.3962	0	0	200.412
1.0	3.3962	0	0	200.412

Questions

How does increasing input delay affect the timing available for internal logic?

Increasing input delay reduces the time available for internal logic to compute before the next clock edge, since the signal is modeled as arriving later at the input pin.

At what input delay does the design first violate timing, if applicable?

The design never violates timing over this sweep (0.1–1.0 ns).

Explain the trend observed in WNS.

WNS stays essentially constant (≈ 3.396 ns) across the whole sweep. This shows the critical path in this design is entirely internal (register-to-register, as confirmed in Part H) rather than starting at a primary input, so changing the input delay barely perturbs the reported WNS.

9 Part F — Output Delay Sweep

Clock period and input delay fixed; only the output-delay constraint was varied.

Output Delay (ns)	WNS (ns)	TNS (ns)	Violating Paths	Area
0.1	3.3959	0	0	200.412
0.2	3.3959	0	0	200.412
0.4	3.3959	0	0	200.412
0.6	3.3959	0	0	200.412
0.8	3.3964	0	0	200.412
1.0	3.3964	0	0	200.412

Questions

What happens to WNS as output delay increases? It stays essentially constant across the sweep (the same internal-critical-path reasoning as Part E applies).

Why does output delay reduce the timing budget available to the internal circuit? The output delay models the setup time the downstream receiving circuit needs, so it must be subtracted from the available clock period, leaving less budget for the internal logic to settle before the data is captured externally.

Compare the input-delay and output-delay experiments. In both sweeps WNS remained essentially unchanged, confirming the critical path of this design is entirely internal (register-to-register); external I/O delay constraints therefore have negligible effect on WNS for this particular design.

10 Part G — Clock Uncertainty Sweep

Supplied SDC used; clock uncertainty varied while other constraints were held fixed.

Clock Uncertainty (ns)	WNS (ns)	TNS (ns)	Violating Paths	Area
0	3.4059	0	0	200.412
0.05	3.3559	0	0	200.412
0.10	3.3059	0	0	200.412
0.20	3.2059	0	0	200.412
0.30	3.1059	0	0	200.412

Questions

Why does increasing clock uncertainty reduce timing margin? Clock uncertainty is subtracted directly from the available clock period as a guard-band against jitter and skew, so the tool must assume the effective period is shorter, which reduces the reported slack one-for-one with the uncertainty added (WNS drops by exactly Δ for each Δ increase in uncertainty here).

Does clock uncertainty significantly change the area in your design? Support your

answer with data. No – area remained exactly $200.412\mu\text{m}^2$ across the whole 0–0.30 ns sweep, since the design still has ample positive slack at every point and Genus has no need to resize cells. **What does clock uncertainty represent physically?** It represents two real physical non-idealities: clock jitter (cycle-to-cycle timing variation of the clock source/PLL) and clock skew (the difference in clock arrival time at different flip-flops due to unequal clock-tree wire delays).

11 Part H — Critical Path Analysis

Timing Parameter	Result
Startpoint	u_datapath_working_multiplicand_reg[0]/CK
Endpoint	u_datapath_accumulated_prod_reg[7]/D
Launch clock	clk
Capture clock	clk
Data path delay	1465 ps
Arrival time	1465 ps
Required time	4820 ps
WNS (at 5.0 ns period)	$4820 - 1465 = 3355\text{ ps} \approx 3355.9\text{ ps}$ (matches Part B)

Critical path (at 5.0 ns period):

u_datapath_working_multiplicand_reg[0]/CK → NAND2X1 → INVX1 → ADDFX1 → ADDFX1 → ADDFX1 → NAND4XL → XNOR2X1 → AOI22X1 → NOR2X1 → u_datapath_accumulated_prod_reg[7]/D

Critical path (at 1.0 ns period, after re-synthesis):

u_controller_current_state_reg[0]/CK → INVX1 → NOR2X1 → NOR2X1 → AOI21X1 → OAI2BB1X1 → OAI211X1 → u_datapath_accumulated_prod_reg[6]/D

Answer

- **Which logic block is responsible for the critical path?** The datapath block – specifically the ripple-carry addition chain of the 8-bit adder feeding the accumulated-product register (consistent with the adder architecture described in **Lab 1, Section 2 (“Datapath Elements”)**).
- **Which cell or group of cells contributes significantly to the delay?** The ADDFX1 (full-adder) cells in the ripple-carry chain.
- **Does the critical path change when the clock constraint changes?** Yes – at the relaxed 5.0 ns constraint the critical path runs through the datapath adder chain, whereas at the aggressive 1.0 ns constraint the critical path shifts to the controller’s next-state logic feeding back into the datapath, since Genus has already resized/optimized the original datapath-adder path.

12 Part I — Area and Cell Usage Analysis

Cell Type	Instances	Area/Cell	Total Area
DFFHQX1	13	5.472	71.136
SDFFQX1	4	7.524	30.096
ADDFX1	3	5.130	15.390
AOI22X1	6	2.052	12.312
NAND2X1	11	1.026	11.286

Answer

- **Which cell type is used most frequently?** DFFHQX1 (13 instances).
- **Which cell type contributes the most area?** DFFHQX1 ($71.136 \mu\text{m}^2$).
- **How many sequential cells are present?** 18 (matches Part B: 13 DFFHQX1 + 4 SDF-FQX1 + 1 additional flop).
- **How many combinational cells are present?** 58 (baseline, 200 MHz).
- **Does the cell composition change when a more aggressive timing constraint is used?** Yes – the leaf-instance count rises from 76 (200 MHz) to 112 (1000 MHz) as the constraint tightens (see Part C).

13 Part J — Fanout Analysis

Parameter	Result
Maximum fanout	18
Net with maximum fanout	clk
Driver of maximum-fanout net	Primary input port
Average fanout	2.3

Answer

- **Why can high fanout cause timing problems?** A net with high fanout drives many downstream gate inputs, presenting a large capacitive load to its driver; this increases the net's RC delay and can slow down the switching of every path that depends on that net.
- **Does Genus insert buffers or change cell sizes to handle fanout in your design?** Yes – as the clock period is tightened from 5.0 ns to 1.0 ns, Genus inserts additional buffers/upsizes drive cells (instance count rises from 76 to 112) to keep fanout-loaded nets meeting the tighter timing.
- **Compare the fanout results for the relaxed and aggressive timing constraints.** Maximum fanout (18, on clk) and average fanout (2.3) are identical at both the relaxed and aggressive constraints – Genus meets tighter timing by resizing/buffering cells rather than by restructuring the fanout topology itself.

14 Part K — Relaxed vs. Aggressive Constraint Comparison

Metric	Relaxed	Aggressive
Clock period	5.0 ns	1.0 ns
Frequency	200 MHz	1000 MHz
WNS	3.3559 ns	0.0065 ns
TNS	0	0
Violating paths	0	0
Total area	200.412	240.152
Cell count	76	112
Maximum fanout	18	18
Total power	Not available	Not available

Discussion

As the timing requirement becomes more aggressive, Genus performs internal restructuring of the netlist: leaf-instance count rises from 76 to 112 and total area grows from 200.412 to 240.152 μm^2 (a 19.8% increase), as the tool replaces the small ripple-carry adder cells used at the relaxed constraint with faster, larger-drive-strength cells and adds buffering, in order to bring the datapath delay down from 1465 ps to within the 1.0 ns budget. This also shifts the critical path itself, from the datapath adder chain (5.0 ns case, Part H) to the controller’s next-state logic (1.0 ns case, Part H) – direct evidence that the optimization is not merely a uniform up-sizing but a structural re-balancing of delay between the datapath and control path. No power data is available to corroborate a power/area trade-off, since power analysis was not enabled in the supplied flow (see Part B).

15 Part L — Final Design Recommendation

Recommended clock period: 1.3 ns

Recommended frequency: 769.2 MHz

WNS: 0.0952 ns

Area: 200.412 μm^2

Power: Not available (not measured in this flow)

Justification

The 1.3 ns operating point is recommended using three QoR parameters, not WNS alone:

1. **Timing margin:** WNS = 0.0952 ns is comfortably positive, giving headroom against PVT/jitter variation (Part D), unlike the 1.0 ns point where WNS is only 0.0065 ns and effectively at the timing-clean boundary.
2. **Area:** At 1.3 ns the total cell area is 200.412 μm^2 – identical to the relaxed 5.0 ns baseline and well below the 240.152 μm^2 required at 1.0 ns, so no area penalty is paid for the higher frequency at this point (Part C).
3. **Cell count / structural stability:** At 1.3 ns the instance count and critical path have not yet been restructured by the tool the way they are at 1.0 ns (Part H, Part K), indicating a more conservative, lower-risk implementation.

Overall, 1.3 ns/769.2 MHz nearly triples the baseline 200 MHz throughput while retaining the baseline area and a healthy timing margin, making it a better balanced operating point than either the very relaxed 200 MHz baseline (Part B) or the marginal 1000 MHz timing-clean limit (Part D).

TASK 2

16 Part 1 — Exhaustive Simulation of the Design

16.1 Determine the Exhaustive Test Space

The exhaustive test space is defined over the two data operands that determine the functional result, $A = \text{multiplicand_in}[3:0]$ and $B = \text{multiplier_in}[3:0]$, since `start` is simply held high to launch each operation and `reset` is only asserted once at the beginning of the testbench (see Lab 1, Section 8.2).

- Number of independent primary-input bits (data operands): **8** (A : 4 bits, B : 4 bits)
- Number of possible input vectors: $2^8 = \mathbf{256}$
- **Is exhaustive simulation practical for this design?** Yes – 256 is a very small number of vectors, so a fully exhaustive $A \times B$ sweep is trivially practical to simulate.

16.2 Exhaustive Simulation

- Number of vectors simulated: **256** ($A = 0..15$, $B = 0..15$)
- Simulation time: 2 seconds
- Output/reference file generated: QuestaSim transcript, all 256 tests reporting [PASS] (excerpt below, full transcript in Appendix A)

Listing 2: Excerpt of the exhaustive simulation transcript (256/256 PASS)

```

1 # Test: 15 x 13 | Exp Prod: 195, Act Prod: 195 | Exp Cycles: 15, Act Cycles: 15 |
  [ PASS ]
2 # Test: 15 x 14 | Exp Prod: 210, Act Prod: 210 | Exp Cycles: 16, Act Cycles: 16 |
  [ PASS ]
3 # Test: 15 x 15 | Exp Prod: 225, Act Prod: 225 | Exp Cycles: 17, Act Cycles: 17 |
  [ PASS ]
4 #
  =====
5 #                                TESTBENCH COMPLETE
6 #
  =====
7 # ** Note: $finish : tb_top.v(116)
8 #    Time: 34620 ns   Iteration: 1   Instance: /tb_top

```

17 Part 2 — Stuck-at Fault Injection

One internal net was selected from the Genus-synthesized gate-level netlist (`work/top_gates.v`), chosen in the middle of the logic rather than at a primary I/O.

17.1 Fault Selection

- Net name: `n_76`
- Fault type: SA-0 (stuck-at-0)
- Original driver cell: `g24171__1881` (AOI22X1)
- Fanout/load cell: `g24135__6260`

17.2 Fault Injection

- Net modified: `n_76`
- Original driver: `g24171__1881`
- Fault modeling: the driver's original output pin was redirected to a dummy net (`n_76_broken`), and `n_76` was tied permanently to logic 0 (`assign n_76 = 1'b0;`), so the fault site is fully decoupled from its true driver.

Listing 3: Original vs. faulty netlist fragment

```
1 Original:
2   AOI22X1 g24171__1881(.A0(n_73), .A1(n_43), .B0(accumulated_prod[5]),
3                       .B1(n_54), .Y(n_76));
4
5 Faulty:
6   AOI22X1 g24171__1881(.A0(n_73), .A1(n_43), .B0(accumulated_prod[5]),
7                       .B1(n_54), .Y(n_76_broken));
8   assign n_76 = 1'b0;
```

18 Part 3 — Fault Activation and Detection

18.1 Detecting Vector

For input vector all 0's, the good circuit produces output `n_95=0`, while the faulty circuit produces output `n_95=1`.

- Good-circuit value of the faulty net: 1
- Faulty-circuit value: 0
- Primary output(s) showing the error: `n_95`

Why this vector detects the fault: the stuck-at-0 fault is activated because the good circuit drives the faulty net (`n_76`) to 1 for this vector. The fault effect then propagates through the downstream NAND3XL gate (`g24135__6260`) because its other two inputs (`n_68` and `n_80`) are both held at 1 – the non-controlling value for a NAND gate – so the gate's output is fully sensitized to `n_76`. This causes the final output `n_95` to flip from 0 (good) to 1 (faulty), successfully detecting the fault.

19 Part 4 — Bit-Parallel Fault Simulation

The supplied bit-parallel fault simulator (`scripts/faultsim.py`) was run against the Genus-synthesized gate-level netlist to determine a reduced/compacted test set achieving the same fault coverage as exhaustive testing.

Parameter	Result
Total exhaustive vectors (sequential state space)	134,217,728
Compacted test patterns	61

The exhaustive count above is far larger than the 256 combinational input vectors of Part 1 because it enumerates the full sequential state space explored by the fault simulator (internal FSM/counter states $\times$ primary inputs), rather than the primary-input-only space used for the RTL exhaustive functional check. The fault simulator compacts this down to 61 test patterns while retaining the same fault coverage, demonstrating the value of test compaction over brute-force exhaustive testing for a sequential design.

Corrections Made Relative to the Original Submission

While compiling this report, two internal inconsistencies in the originally submitted answers were identified and corrected here using the underlying simulation/synthesis logs as ground truth:

1. **Task 2, Part 1.1:** the primary-input bit count and vector count were originally reported as “9 bits / 512 vectors”. The testbench and transcript evidence (Part 1.2, and **Lab 1, Section 8.3**) show the design is exhaustively exercised over the two 4-bit data operands only (`start` is held high and `reset` pulsed once), giving **8 bits / 256 vectors**, matching the 256-test transcript exactly.
2. **Part D:** “WNS at this point” was originally reported as 0.9935 ns, which does not match any sweep-table entry. The Genus report for the 1.0 ns run (`work/reports/qor_1.0ns.rpt`) shows a critical-path slack of 6.5 ps, i.e. **WNS = 0.0065 ns**, consistent with the Part C/Part D sweep tables; this value is used throughout this report.

A Appendix: Genus Reports

The following reports (referenced throughout this document) are attached in the submission directory `work/reports/`: `top_qor.rpt`, `top_area.rpt`, `top_timing.rpt`, and the per-sweep-point reports (`qor_*.rpt`, `timing_*.rpt`, `area_*.rpt`, `input_*.rpt`, `output_*.rpt`, `uncert_*.rpt`) used to populate Parts C–G.

References

- [1] R. Sarang, *Lab 1 – Deliverables: Sequential Multiplier Using Repeated Addition*, EC5034 VLSI Testing Practice, 30th August 2026.